

Mohammed Suleiman

August 16, 2026

My unit testing approach for the Contact, Task and Appointment features was structured around validating both the object constraints and the service level logic. For every class, I employed boundary value analysis and equivalence partitioning to ensure all of the parameters were thoroughly vetted. My approach was aligned to the requirements. The specifications demanded that fields not exceed certain lengths or be null. I tested these rules. For example, the requirements dictate that an Appointment date cannot be in the past. My tests validate this by passing a historical epoch date and then verifying if the object throws the correct error. Similarly, I tested the maximum boundaries of the strings to make sure requirements like the 50 character limit for descriptions were enforced.

The overall quality of the JUnit tests are effective, which is reflected in the code coverage percentage. By intentionally designing tests for the happy paths and unhappy paths, the tests execute 100% of the conditional branches within the if/else validation statements. I also made sure that the high coverage percentage guarantees that the application handles edge cases gracefully without crashing.

Writing the JUnit tests required careful planning to make certain that the testing code was both reliable and clean. I validated that my tests were technically sound by using JUnit 5 assertions libraries to capture and validate runtime exceptions. Instead of letting the program crash on invalid data, I verified that the code successfully caught the errors. A specific example of this logic is one test verifying that adding a null appointment throws an exception with: `Assertions.assertThrows(IllegalArgumentException.class, () -> service.add(null))`.

To make the tests efficient, I utilized setup annotations and constant variables. I used the `@BeforeEach` annotation to instantiate a new service object before every test run, guaranteeing a clean state. For example, `service = new TaskService()` is triggered automatically before each test. Additionally, I declared global constant variables like `private final String VALID_ID_MAX = "1234567899"` so I could reuse boundary strings without hardcoding them constantly.

The primary software testing techniques I employed were boundary value analysis and equivalence partitioning. Boundary value analysis involves testing the exact edges of acceptable input. Equivalence partitioning involves grouping inputs into valid and invalid buckets such as testing standard text versus null values. I didn't use integration or performance testing for this project. Integration testing involves how different system modules interact with one another, like connecting the service classes to a database or UI. Performance testing checks how the app behaves under heavy user traffic. Unit testing and boundary analysis are highly practical in the early stages of any software development lifecycle to confirm fundamental building blocks are robust. In contrast, integration testing is necessary later in development when connecting to API's or GUI. Performance testing is crucial before deploying web apps or mobile apps to production to verify the servers can handle the expected user base without timing out.

Operating as a software tester required adopting a highly cautious and skeptical mindset. I had to assume that the user or the external system might provide the worst possible data. It was important to appreciate the interrelationships of the code because the classes are deeply connected. A failure at the base object level would cascade up and corrupt the data within the `ContactService` list in memory. I limited my bias by relying solely on the rubric requirements to write my test cases rather than relying on how I expected the code to run. When developers test

their own code, bias is a big concern. A developer will test the exact happy path they programmed the software to handle, tying them to edge cases or user behaviors.

Maintaining discipline in software engineering is vital for long term product quality. Cutting corners on testing leads directly to technical debt. Technical debt then means that bugs will inevitably surface in the production environment where they are more complicated to track and fix. As a practitioner in the field, I plan to avoid technical debt by prioritizing test coverage as well as with feature development rather than treating it like an afterthought.